hystrix

1.创建command

HystrixCommand 主要用于仅仅会返回一个结果的调用。
HystrixObservableCommand 主要用于可能会返回多条结果的调用。

2.调用command方法

execute()：调用后直接 block 住，属于同步调用，直到依赖服务返回单条结果，或者抛出异常。
queue()：返回一个 Future，属于异步调用，后面可以通过 Future 获取单条结果。
observe()：订阅一个 Observable 对象，Observable 代表的是依赖服务返回的结果，获取到一个那个代表结果的 Observable 对象的拷贝对象。
toObservable()：返回一个 Observable 对象，如果我们订阅这个对象，就会执行 command 并且获取返回结果。

3.检查是否开启缓存

requestCache.enabled参数

4.检查是否开启了断路器

检查这个 command 对应的依赖服务是否开启了断路器。如果断路器被打开了，那么 Hystrix 就不会执行这个 command，而是直接去执行 fallback 降级机制，返回降级结果。

5.检查线程池/队列/信号量是否已满

如果这个 command 线程池和队列已满，或者 semaphore 信号量已满，那么也不会执行 command，而是直接去调用 fallback 降级机制，同时发送 reject 信息给断路器统计。

6.执行 command

调用 HystrixObservableCommand 对象的 construct() 方法，或者 HystrixCommand 的 run() 方法来实际执行这个 command。
如果是采用线程池方式，并且 HystrixCommand.run() 或者 HystrixObservableCommand.construct() 的执行时间超过了 timeout 时长的话，那么 command 所在的线程会抛出一个 TimeoutException，这时会执行 fallback 降级机制，不会去管 run() 或 construct() 返回的值了。另一种情况，如果 command 执行出错抛出了其它异常，那么也会走 fallback 降级。这两种情况下，Hystrix 都会发送异常事件给断路器统计。

7.断路健康检查

Hystrix 会把每一个依赖服务的调用成功、失败、Reject、Timeout 等事件发送给 circuit breaker 断路器。断路器就会对这些事件的次数进行统计，根据异常事件发生的比例来决定是否要进行断路（熔断）。如果打开了断路器，那么在接下来一段时间内，会直接断路，返回降级结果。

如果在之后，断路器尝试执行 command，调用没有出错，返回了正常结果，那么 Hystrix 就会把断路器关闭。

8.调用 fallback 降级机制

断路器处于打开状态；
线程池/队列/semaphore满了；
command 执行超时；
run() 或者 construct() 抛出异常。

HystrixCommand 中，实现 getFallback() 方法，可以提供降级机制。
HystrixObservableCommand 中，实现 resumeWithFallback() 方法，返回一个 Observable 对象，可以提供降级结果。
如果没有实现 fallback，或者 fallback 抛出了异常，Hystrix 会返回一个 Observable，但是不会返回任何数据。
不同的 command 执行方式，其 fallback 为空或者异常时的返回结果不同。

对于 execute()，直接抛出异常。
对于 queue()，返回一个 Future，调用 get() 时抛出异常。
对于 observe()，返回一个 Observable 对象，但是调用 subscribe() 方法订阅它时，立即抛出调用者的 onError() 方法。
对于 toObservable()，返回一个 Observable 对象，但是调用 subscribe() 方法订阅它时，立即抛出调用者的 onError() 方法。

正常执行代码流程

1. HystrixCommand类 execute()方法
2. HystrixCommand类 queue()方法
3. AbstractCommand类 toObservable()方法
4. AbstractCommand类 applyHystrixSemantics(final AbstractCommand<R> _cmd)方法
5. AbstractCommand类 executeCommandAndObserve(final AbstractCommand<R> _cmd)方法
6. AbstractCommand类 executeCommandWithSpecifiedIsolation(final AbstractCommand<R> _cmd)方法
7. AbstractCommand类 getUserExecutionObservable(final AbstractCommand<R> _cmd) 方法
8. HystrixCommand 类 getExecutionObservable()方法
9. HystrixCommand类 run()方法

异常执行代码流程

1. short-circuited短路(circuitBreaker.allowRequest()) ：AbstractCommand类 handleShortCircuitViaFallback()方法
2. threadpool-rejected线程池拒绝(RejectedExecutionException)：AbstractCommand类 handleThreadPoolRejectionViaFallback(e)方法
3. semaphore-rejected信号量拒绝(executionSemaphore.tryAcquire())：AbstractCommand类 handleSemaphoreRejectionViaFallback()方法
4. timed-out超时(HystrixTimeoutException)：AbstractCommand类 handleTimeoutViaFallback()方法
5. failed执行失败：AbstractCommand类 handleFailureViaFallback(e)方法